HARD BUDGETS EXPOSE THE REPAIR–BREAKAGE TRADEOFF IN AGENT WORKFLOWS

Anonymous authors
Paper under double-blind review

ABSTRACT

Whether structured agent workflows—generate, verify, refine—beat a single model call is actively disputed: some studies report large gains, others report that the gains vanish once inference compute is equalised. We show that both findings are correct and describe the same phenomenon seen from opposite sides. Under a reservation-based ledger that enforces per-instance caps on tokens, calls, tool uses, latency and dollars *before* each call is issued, we decompose a workflow's net effect into two opposing terms: *repair*, the rate at which it rescues tasks the baseline fails, and *breakage*, the rate at which it destroys tasks the baseline already solves. On frozen test splits, a standard verify–refine workflow repairs 20.5% of failures and breaks 14.0% of successes, netting -3.3 points while spending $5.5\times$ the dollars. The decomposition is an exact identity, not a fitted model, and it yields a design rule that we verify causally: protecting the incumbent answer —never overwriting a temperature-0 draft unless a verifier rejects it—drives breakage to 0.000 in all twelve family$\times$budget cells, turning a net-zero workflow into a small but reliable gain. Degrading the verifier moves the tradeoff exactly as the decomposition predicts: with visible tests removed the same workflow loses 11.8 points. To rule out that incumbent protection is an artifact of our own design intuitions, we run a budget-safe automated search; in the code family it independently rediscovers the same shape, and a single budget-contingent policy matches per-budget static search—including at a budget never seen during search—using half the search cost. We report our preregistered claims as they landed, including a locked out-of-domain prediction that failed and the design confound that caused it.

1 INTRODUCTION

Agent systems are routinely built as workflows: a model drafts an answer, a verifier inspects it, a refiner revises it, and the loop repeats. Whether this machinery is worth its cost is contested. One line of work reports that multi-agent and multi-step structures dominate single-call baselines on the compute–accuracy frontier; another argues, with information-theoretic backing, that the reported advantages disappear once inference compute is genuinely matched, and are artifacts of unaccounted computation.

The dispute has persisted because both sides report a *net* number. We argue that the net number is the wrong unit of analysis. A workflow that adds verification and refinement does two things at once: it rescues tasks the baseline would have failed, and it damages tasks the baseline had already solved. These two effects are of comparable magnitude in practice, and their difference—the quantity everyone reports—is small and unstable. Which sign it takes depends on the operating point.

We make this precise and then act on it. Our contributions:

1. **A hard-budget protocol that makes the comparison well posed.** We enforce per-instance budgets with a reservation ledger: a node may execute only if its worst-case billable cost can be reserved against the remaining caps, otherwise control flow is routed to a safe exit. Overruns become impossible rather than merely reported, so every comparison is genuinely compute-matched (§3).

2. **The repair–breakage decomposition.** We show that a workflow's net effect over a baseline is exactly $(1-p)r_{\text{eff}} - p\, b_{\text{eff}}$ where p is baseline accuracy, r_{eff} the repair rate on baseline failures and b_{eff} the breakage rate on baseline successes. This is an algebraic identity for binary outcomes, not a model fit — a distinction we make explicit, because it means the decomposition is *exhaustive* but cannot be "validated" on the data used to estimate its terms (§4).
3. **Causal control of both terms.** Degrading verifier signal moves breakage and, with it, the net effect: removing visible tests turns a -3.3-point workflow into a -11.8-point one. Conversely, an *incumbent-protecting* construction drives breakage to 0.000 in every cell we measured, converting a net-zero workflow into a reliable if modest gain (§5).
4. **Automated search as a bias check, not as a proposed optimiser.** We build a budget-safe searcher over a restricted workflow DSL. Its role is to test whether incumbent protection is an artifact of our own design intuitions. In the code family the search rediscovers the same shape unprompted, and one budget-contingent policy matches per-budget static search—at both searched budgets and at an intermediate budget it never saw—for half the search cost. In the math family it does not (§6).
5. **Preregistration, including the parts that failed.** All confirmatory claims were written and git-tagged before the corresponding frozen splits were touched. We report them as they landed: one Part-I claim was refuted, three Part-II claims were not supported, and a locked out-of-domain prediction failed because of a confound in our own experiment design, which we diagnose and then decouple (§7, §8).

2 Related work

Automated workflow design. A body of work searches over agent structures: representing agents as optimisable graphs, evolving code-level workflows with tree search, composing modular designs, and evolving populations of heterogeneous workflows under multi-objective criteria. These methods optimise *within* a design space; our question is prior to that—what determines whether any structure in that space is worth its cost. Our searcher is deliberately unremarkable, because its job is falsification, not performance.

Budget-aware inference. Recent work conditions inference-tree node selection on remaining resources, and makes workflow *search* cost-aware via reuse and caching. Our budget conditioning acts at the level of workflow control flow rather than tree node selection, and our ledger enforces budgets before execution rather than accounting for them afterwards.

Test-time compute allocation. The sampling-versus-refinement crossover is documented for single-model test-time compute: which strategy wins depends on per-sample accuracy and verifier quality. We lift the question to workflow architecture, place it under a five-dimensional hard-budget protocol, and—most importantly—identify the mechanism (breakage) that the net-effect framing hides.

The dispute we address. Recent studies reach opposite conclusions about whether multi-agent structure improves the compute–accuracy frontier. Our decomposition reconciles them without adjudicating either as wrong.

3 Setup: hard budgets and a reservation ledger

Budget profiles. A deployment budget is a vector of caps on input tokens, output tokens, LLM calls, tool calls, wall-clock seconds, and dollars. We use five profiles spanning \$0.02–\$0.25 per task; the two searched profiles are `tight` (\$0.10, 4 calls) and `loose` (\$0.25, 8 calls), and `unseen` (\$0.15) is touched only at final evaluation and is inaccessible to every search, selection and tuning code path.

Reservation ledger. Before a node executes, its worst-case billable vector is computed and *reserved* against the remaining caps. If the reservation fails the node does not run; control flow fol-

lows a `reserve_feasible` fallback or terminates with the current answer. After the call, actual usage is settled and the surplus released. An overrun at settlement is treated as a runtime defect and surfaced, not absorbed. A 3,000-workflow fuzz sweep over random legal workflows and random profiles produced zero cap violations in any dimension.

This matters for the science, not just for hygiene. Under post-hoc accounting, a workflow that overspends is recorded as expensive; under reservation, it is prevented, and the prevention itself becomes an observable (we report reserve rejections separately). Compute matching is thereby enforced rather than audited.

Verifier taxonomy. We distinguish two verifier regimes and never conflate them. In the *code* family, the workflow's verifier runs the task's own visible tests—environment feedback that is part of the task specification. Hidden extended tests exist only in the external grader. In the *math* and *logic* families, no oracle is available to the workflow: the verifier is a gold-free self-consistency check that re-derives the answer independently and compares. Gold answers never enter workflow state in any family.

Tasks and splits. Code: MBPP+ (120 search / 40 validation / 150 frozen test), with HumanEval+ held out entirely. Math: MATH levels 4–5 over four subjects (120/40/150), with two further subjects held out. Splits were generated once with a fixed seed, hashed, and never regenerated. All final numbers use three execution seeds, seed-averaged per task, with stratified paired bootstrap intervals over 10,000 resamples.

4 THE REPAIR–BREAKAGE DECOMPOSITION

Let B be a baseline and W a workflow evaluated on the same tasks under the same budget. Write $p = \Pr[B \text{ correct}]$, and define

$$r_{\text{eff}} = \Pr[W \text{ correct} \mid B \text{ incorrect}], \qquad b_{\text{eff}} = \Pr[W \text{ incorrect} \mid B \text{ correct}]. \tag{1}$$

Then the net change in accuracy is

$$\Delta \;=\; \Pr[W] - \Pr[B] \;=\; (1-p)\, r_{\text{eff}} \;-\; p\, b_{\text{eff}}. \tag{2}$$

This is an identity, and saying so matters. For binary per-task outcomes, equation 2 holds exactly by the law of total probability. It therefore cannot be "validated" by estimating $r_{\text{eff}}, b_{\text{eff}}$ on a dataset and checking that it reproduces Δ on that same dataset — that check is tautological and we do not present one. What the identity buys is different and, we think, more useful: it guarantees the decomposition is *exhaustive*. Every effect a workflow has on accuracy is either repair or breakage; there is no third channel. A literature that reports only Δ is therefore reporting a difference of two quantities that can each be large.

A design rule follows. Rearranging equation 2, a workflow helps iff

$$\frac{b_{\text{eff}}}{r_{\text{eff}}} \;<\; \frac{1-p}{p}. \tag{3}$$

Two regimes follow immediately. With an oracle verifier that never rejects a correct answer, $b_{\text{eff}} \to 0$ and the workflow can only help. With a verifier carrying no signal, every answer is flagged, b_{eff} rises to the unconditional rate at which refinement damages correct answers, and equation 3 is violated whenever the baseline is strong. *Incumbent protection* is the constructive way to reach $b_{\text{eff}} \approx 0$ without an oracle: emit a temperature-0 draft, and replace it only if a verifier actively rejects it.

What does and does not transfer. We stress-test transfer of $r_{\text{eff}}, b_{\text{eff}}$ across conditions in §7. It largely fails for unconstrained workflows: both parameters drift with domain and budget, and their difference drifts with them. It succeeds for incumbent-protecting workflows, where b_{eff} is pinned to zero by construction and only r_{eff} must transfer. Constraining a system makes it predictable; this is a claim about our construction, not a general law.

Table 1: Frozen test splits, 150 tasks per family, 3 execution seeds. Baselines are the strongest single-call structure per family (`direct` for code, `cot` for math). Intervals are stratified paired bootstrap 95% CIs. *vanilla* is generate→verify→refine with up to 3 iterations; *protected* is the same loop with a temperature-0 incumbent that is replaced only on verifier rejection.

Family	Budget	Workflow	Δ (95% CI)	repair	breakage	$/task
code	tight	vanilla	-0.104 $[-0.162, -0.047]$	0.162	0.224	0.0062
code	tight	protected	$+0.013$ $[+0.002, +0.031]$	0.034	**0.000**	0.0022
code	unseen	vanilla	-0.038 $[-0.093, +0.018]$	0.214	0.150	0.0085
code	unseen	protected	$+0.020$ $[+0.002, +0.042]$	0.060	**0.000**	0.0025
code	loose	vanilla	-0.033 $[-0.087, +0.020]$	0.205	0.140	0.0090
code	loose	protected	$+0.020$ $[+0.002, +0.042]$	0.060	**0.000**	0.0025
math	tight	vanilla	$+0.004$ $[-0.029, +0.038]$	0.126	0.052	0.0076
math	tight	protected	$+0.000$ $[+0.000, +0.000]$	0.000	**0.000**	0.0075
math	unseen	vanilla	$+0.009$ $[-0.022, +0.040]$	0.117	0.042	0.0141
math	unseen	protected	$+0.004$ $[+0.000, +0.011]$	0.018	**0.000**	0.0141
math	loose	vanilla	$+0.016$ $[-0.016, +0.047]$	0.117	0.038	0.0153
math	loose	protected	$+0.011$ $[+0.002, +0.022]$	0.036	**0.000**	0.0153

5 Part I: Confirmatory Experiments

Five claims (C1–C5) were written, committed and git-tagged (`prereg-freeze-partI`) before any frozen test split was executed. We report all five as they landed.

C1 (net-zero), supported. At `loose`, vanilla verify–refine differs from the baseline by -0.033 $[-0.087, +0.020]$ in code and $+0.016$ $[-0.016, +0.047]$ in math, while costing $5.5\times$ and $2.2\times$ the dollars respectively. The workflow buys nothing measurable at several times the price. The decomposition shows why: in code it repairs 20.5% of failures and breaks 14.0% of successes.

C2 (budget floor), supported. At `tight`, code vanilla is -0.104 $[-0.162, -0.047]$: significantly harmful. Structure has an entry fee. When the budget cannot fund enough refinement iterations, repair falls ($0.205 \rightarrow 0.162$) while breakage rises ($0.140 \rightarrow 0.224$), and equation 3 flips. Reserve rejections make the mechanism visible: 35 of 150 tasks at `tight` could not fund a planned refinement call.

C3 (verifier dose–response), supported. Masking visible tests at $1.0 \rightarrow 0.5 \rightarrow 0.0$ moves the code/`loose` effect monotonically: -0.033, -0.033, -0.118 $[-0.178, -0.058]$. With no signal every answer is flagged, breakage climbs from 0.140 to 0.234, and refinement becomes actively destructive. Because masking changes only the verifier, this is a causal manipulation of the b_{eff} term.

C4 (incumbent protection), supported. Breakage is 0.000 in all twelve family×budget cells. In code the protected workflow is significantly positive at all three budgets ($+0.013$, $+0.020$, $+0.020$); in math it is non-inferior everywhere and significantly positive at `loose` ($+0.011$ $[+0.002, +0.022]$). Protection also costs less than vanilla refinement ($0.0025 vs $0.0090 per task at code/`loose`) because most tasks exit at the first verifier pass.

C5 (repair is verifier-type-bounded), refuted. We predicted that oracle-verified repair would strictly exceed non-oracle repair, with the math interval containing zero. Observed: code repair 0.060 $[0.000, 0.137]$, math repair 0.036 $[0.009, 0.072]$. The intervals overlap and the math interval *excludes* zero. Gold-free self-consistency checking produces real repair. We make no claim about verifier-type-bounded repair beyond these intervals.

6 Part II: Automated Search as a Bias Check

Incumbent protection was derived by us, from our own decomposition, and then tested by us. That is a route by which a design intuition can be mistaken for a finding. We therefore built a budget-safe

searcher whose purpose is to check whether an optimiser with no knowledge of our reasoning arrives at the same place.

Searcher. Candidates are workflows in a restricted DSL (at most 8 nodes, at most 2 loops of at most 3 iterations, a fixed node whitelist, no model-generated executable code, static validation before any spend). Search is evolutionary with multi-fidelity racing over nested task subsets, evaluated jointly at both searched budgets, ranked by a score that averages per-budget accuracy with the worst budget to prevent single-budget speculation. The only difference between the budget-contingent arm (HBWS) and the static control is whether edges may condition on remaining budget; operator sets are otherwise identical, and we verified over 200 sampled workflows that the static arm never emits a budget predicate. Protocol A gives the static control a full search budget *per budget tier*, i.e. twice what HBWS receives.

Two bugs that would have invalidated this section. Our first search runs were statistically void, and we report the diagnosis because the failure mode is easy to reproduce. First, the racing rule compared a candidate's lower confidence bound against the incumbent's with a fixed slack of 0.10; the Hoeffding half-width difference between the $n{=}24$ and $n{=}64$ rungs is 0.107, so promotion from the cheapest rung was arithmetically impossible and 188 of 189 candidates never left it. Racing must compare a candidate's *upper* bound against the incumbent's lower bound. Second, our pre-call token estimate (characters/3) under-counted punctuation-dense code prompts by $\sim$55%, producing 124 settlement overruns that were scored as infeasible candidates. After both fixes, promotion rose from 0.5% to 87.5% and infeasible candidates fell to zero.

Convergence (dev, exploratory). At full fidelity, every search arm—budget-contingent and both static controls—selected the same shape: a temperature-0 first draft, a verifier gate, and refinement only on rejection. The budget-contingent arm additionally gated refinement on remaining budget (thresholds 0.6; one variant used a two-stage $0.3/0.6$). An optimiser with no access to our theory reconstructed the theory's prescription.

Confirmatory results (frozen test), reported as they landed. Five claims (P1–P5) were tagged `prereg-freeze-partII` before any searched policy touched validation or test.

- **P1 (non-inferiority at both budgets), not supported.** Code: $+0.013$ (LCB -0.001) at `tight` and $+0.001$ (LCB -0.010) at `loose`, both within the $\delta = 0.03$ margin. Math: $+0.049$ $[+0.009, +0.091]$ at `loose`—significantly *better*—but -0.022 (LCB -0.049) at `tight`, outside the margin. P1 required all four cells.
- **P2 (cost reduction $\geq 20\%$), not supported.** Code achieved 10.5%. Math was 191% more expensive: the selected budget-contingent policy verifies and refines while the selected static policy is a bare generate. This is a Pareto trade ($+0.049$ accuracy at $3\times$ cost), not a cost win.
- **P3 (unseen \$0.15 budget), supported in code only.** Code: -0.001 $[-0.013, +0.010]$ against the preregistered static transfer control at a budget no search ever saw. Math: -0.011 (LCB -0.040), just outside the margin.
- **P4 (shape convergence), weaker than on dev.** Only 3 of 12 selected policies show the protected shape, versus every arm on dev. The preregistered one-shot selection uses a 40-task validation split whose selection variance is large relative to the 1–2 point differences between candidates. We report this as a protocol defect rather than adjusting it.
- **P5 (economics).** In code, HBWS costs \$60 to search against \$120 for per-budget static search, and deploys \$0.00029/task cheaper: it dominates from the first task, $N^\star = 0$. In math, deployment is more expensive and $N^\star = \infty$.

What we take from this. Not that our searcher is good. The defensible readings are: (i) an independent optimiser reconstructs incumbent protection in the code family, which is evidence that the principle is not our design bias; (ii) one policy can cover multiple budgets, including an unseen one, at half the design cost, in the code family; (iii) it does not transfer to math, where the searched policies trade cost for accuracy instead.

Table 2: Breakage of the incumbent-protecting workflow across verifier regimes and domains. The guarantee tracks verifier signal, not domain. The single violation is the one condition without signal, and restoring signal on the *same tasks* restores the guarantee.

Condition	Verifier	breakage	Δ (95% CI)
in-domain code (MBPP+)	oracle tests	0.000	$+0.020$ $[+0.002, +0.042]$
in-domain math (MATH)	non-oracle check	0.000	$+0.011$ $[+0.002, +0.022]$
out-of-domain math (held-out subjects)	non-oracle check	0.000	$+0.003$ $[-0.013, +0.023]$
out-of-domain code (HumanEval+)	*none*	0.114	-0.087 $[-0.140, -0.033]$
out-of-domain code, signal restored	recovered tests	0.000	$+0.005$ $[+0.000, +0.015]$

7 External validity

A locked prediction that failed. Before computing any out-of-domain effect, we committed (tag `ood-predictions-locked`) predictions for HumanEval+ and held-out math subjects, transferring $r_{\text{eff}}, b_{\text{eff}}$ from the in-domain frozen test split. The prediction failed: mean absolute error 0.061, sign agreement $6/8$, and the structural breakage bound ($b_{\text{eff}} \leq 0.02$ for protected workflows) was *violated* in code, at 0.125 and 0.114.

Diagnosis: our confound, not the framework's. Our OOD code split had been constructed with empty visible tests. Its verifier therefore ran in the no-signal regime while the transferred parameters came from the oracle regime: the test varied domain *and* verifier availability at once. Substituting matched no-signal parameters does not rescue the quantitative prediction either (errors 0.09–0.15), so the quantitative transfer claim fails on its own terms and we do not make it.

Decoupling. We recovered visible tests for the same OOD tasks by parsing docstring examples ($68/100$ tasks), yielding a condition that differs from in-domain only in domain. Table 2 shows the result: breakage returns to 0.000.

What transfers. In the decoupled condition, the transferred prediction is accurate for protected workflows (errors 0.002 and 0.000) and badly wrong for vanilla ones (errors 0.169 and 0.108). This is the constrained-versus-unconstrained asymmetry of §4: pinning b_{eff} to zero removes one of two drifting parameters.

Prospective validation in a third domain. The tests above are retrospective: the domains were chosen, then the framework was applied. To test the framework prospectively we selected a third domain that no part of this project had touched—BIG-Bench Hard logical deduction, shuffled-object tracking and date understanding (120 tasks, multiple choice, exact-match grading, a gold-free self-consistency verifier)—wrote down what the framework predicts, committed it under tag `prospective-locked`, and only then ran anything. The predictions were deliberately structural, since §7 had already shown that $r_{\text{eff}}, b_{\text{eff}}$ do not transfer quantitatively: **PV1**, incumbent-protecting breakage ≤ 0.02 at both budgets; **PV2**, vanilla breakage materially higher (> 0.02 and $\geq 3\times$); **PV3**, incumbent-protecting effect ≥ -0.01 (never materially harmful); and **PV4**, explicitly no prediction about repair magnitude or absolute accuracy.

PV1 holds in all four cells (0.000, 0.006, 0.000, 0.003): the breakage guarantee, stated in advance, survived transfer to a domain with a different task format, a different answer type and a much stronger baseline. **PV3 holds** in all four cells. **PV2 does not**: at `loose`, vanilla breakage is 0.006—no worse than protected. The reason is visible in the same table and is a boundary we did not anticipate. Breakage requires that refinement be *able* to destroy a correct answer. Here answers are single multiple-choice labels and the baseline is right 94% of the time, so an unprotected refinement pass has little to damage; at `tight`, where refinement acts under pressure, the gap reappears (0.026–0.029 vs 0.000). Incumbent protection is insurance, and its premium is only visible where the risk is real.

We regard this as the strongest evidence in the paper for the mechanism, and also as its sharpest limit: the structural prediction transferred prospectively, the comparative prediction did not, and the failure localises exactly when the construction is worth adopting.

Table 3: Prospective validation on BIG-Bench Hard (120 tasks, 3 seeds), a domain untouched by any search, tuning or exploratory run. Predictions were locked before execution. This is a demanding regime: the baseline is strong ($p = 0.919$–0.944), so the threshold of equation 3 is only $(1-p)/p = 0.088$ and even small breakage should flip the sign.

Budget	Workflow (vs. own first draft)	Δ (95% CI)	breakage	repair
tight	vanilla (draft `direct`)	$+0.000\ [-0.031, +0.033]$	0.026	0.200
tight	protected (draft `direct`)	$+0.000\ [+0.000, +0.000]$	**0.000**	0.000
tight	protected (draft `cot`)	$+0.000\ [+0.000, +0.000]$	**0.000**	0.000
loose	vanilla (draft `direct`)	$+0.025\ [-0.000, +0.056]$	0.006	0.200
loose	protected (draft `direct`)	$+0.025\ [+0.003, +0.050]$	**0.006**	0.200
loose	protected (draft `cot`)	$+0.014\ [+0.000, +0.028]$	**0.003**	0.000

8 LIMITATIONS

We state these at greater length than the results, because several of them bound the claims directly.

One model. Every number comes from a single frozen deployment (GPT-4o, fixed API version). Both p and r_{eff} depend on model quality; b_{eff} plausibly does too. Nothing here establishes that the numerical tradeoff transfers across model scales, and our own transfer tests show $r_{\text{eff}}, b_{\text{eff}}$ are not stable even across domains for unconstrained workflows.

The identity is not a prediction. equation 2 is exact for binary outcomes. Its scientific content is exhaustiveness plus the design rule equation 3; it makes no forecast that could be falsified on the data used to estimate its terms. Our only genuine transfer tests are those in §7, and one of them failed.

Selection variance in Part II (P4). Our preregistered rule selects a policy once on a 40-task validation split. Candidate policies differ by 1–2 points; a 40-task split cannot resolve that. This is why shape convergence is $12/12$ on dev-set analysis but $3/12$ among selected policies. The correct fix is a larger validation split or repeated selection, both of which we chose not to retrofit after the freeze.

Math family boundary in Part II. The budget-contingent policy is significantly better at math/`loose` and worse at math/`tight`. We do not have a mechanism-level explanation supported by data. It would be easy to narrate this as a boundary condition predicted by equation 3, and we checked: at math/`tight` the measured ratio is $b_{\text{eff}}/r_{\text{eff}} = 0.41$ against a threshold $(1-p)/p = 0.43$, i.e. the rule predicts a (marginal) *benefit*, and the measured structure-vs-baseline effect is indeed $+0.004$. The rule therefore does not explain the policy-vs-policy result, and we do not claim it does.

Small hard-task strata out of domain. On OOD code with signal restored, the baseline solves 62 of 68 tasks, leaving 5 hard tasks. The repair estimate there (0.067) is correspondingly noisy, though the breakage estimate rests on the 62-task stratum.

Restricted design space. Our DSL admits at most 8 nodes, 2 loops and a fixed node whitelist, and forbids model-generated executable code. Results about "structure" are results about this space. Richer spaces may contain workflows whose repair–breakage profile we cannot express.

Verifier taxonomy is coarse. We treat verifiers as oracle, non-oracle-with-signal, or no-signal. Real verifiers vary continuously in sensitivity and specificity, and equation 2 folds both into r_{eff} and b_{eff} rather than separating them. Separating them would give a sharper rule and is the obvious next step.

9 CONCLUSION

The question "does agent workflow structure help?" has no budget-free answer, and reporting a net effect hides the reason. Structure repairs and structure breaks; under matched hard budgets these are of comparable size, so the net effect is small, unstable, and reads differently depending on where in the budget range one measures. Once decomposed, the design question becomes concrete: make breakage impossible. Protecting the incumbent answer does exactly that—breakage 0.000 in every cell we measured, across two verifier types and two domains—and it is what an independent optimiser converges to when asked to search the space itself. The construction fails precisely where the mechanism says it must: when the verifier has no signal to gate on.

REPRODUCIBILITY STATEMENT

All splits are hashed and frozen; all confirmatory claims are git-tagged before the corresponding split is executed (`prereg-freeze-partI`, `prereg-freeze-partII`, `ood-predictions-locked`, `prospective-locked`). The anonymised repository contains the DSL and its validator, the reservation ledger with its fuzz harness, every search registry with parent/mutation lineage, all raw per-task traces, the preregistration with its append-only deviation log including the refuted claims, and a single command that recomputes every table from raw logs.